CS 180 AI Group Project

Semester: 2nd Semester, 2025-2026

Theme: AI App for Everyday Life

1. Overview

Students are required to design, develop, and document an AI-powered application that solves a practical problem in daily life. The goal is to move beyond theoretical models and create a functional tool that demonstrates the intersection of user experience (UX) and intelligent backend processing.

2. Project Scope

The application must address a specific "everyday life" scenario. Examples include, but are not limited to:

- **Personal Productivity:** Smart scheduling or intelligent note-taking.
- **Health & Wellness:** AI nutrition tracking via image recognition, or personalized posture correction using computer vision.
- **Sustainability:** Waste segregation assistance, or smart energy consumption analytics.
- **Accessibility:** Real-time scene description for visually impaired users.

2.1. Scope

- Developing a functional prototype (Web App, Mobile App, or Desktop GUI).
- Integrating at least one core AI capability (Machine Learning, Natural Language Processing, Computer Vision).
- Documenting the data pipeline and model training process.

3. Technical Requirements

3.1. AI Capabilities

The application must utilize at least one of the following technologies (but not limited to):

- **Computer Vision (CV):** Object detection, image classification, OCR.
- **Natural Language Processing (NLP):** Sentiment analysis, summarization, chatbots, language translation.
- **Machine Learning (ML):** Predictive analytics, recommendation engines, clustering.

3.2. Data Requirements

- **Data Sourcing:** Students must identify a dataset (publicly available, not synthetic) used for training/testing.
- **Preprocessing:** Documented steps for data cleaning, normalization, and augmentation.

3.3. Performance Metrics

Students must define and measure success using appropriate metrics:

- **Model Performance:** such as Accuracy, Precision, Recall, F1-Score, or Mean Squared Error
- **System Performance:** Inference time (latency) must be acceptable for a real-time application (e.g., < 2 seconds).

4. Functional Requirements (Features)

These requirements ensure the application is user-friendly and truly "intelligent."

FR1: Intuitive User Input

The application must allow users to provide data in a natural way for the problem being solved.

- **For Computer Vision:** An interface to upload a photo from a gallery or take a live photo using the camera.
- **For NLP:** A text box for user input, or a microphone button for voice-to-text.
- **Requirement:** The input method must be clearly labeled and easy to use for non-technical users.

FR2: Real-time AI Processing

When the user submits input, the backend must process it using the trained AI model immediately.

- **Latency Constraint:** The time between user submission and the AI output appearing must be **under 2 seconds**.
- **Robustness:** The system must not crash if the input is improper (e.g., if a user uploads a non-image file to an image classifier).

FR3: Actionable Insight Display

The AI output cannot just be a raw number (like a probability score) or raw text. It must be translated into information the user can act upon.

- **Example (Bad):** "Probability of apple: 0.95"
- **Example (Good):** "This is an apple. It is safe to eat. [Calories: 95]"
- **Format:** Use clear text, graphics, or cards to display results.

FR4: Personalization (Context Awareness)

The system should adapt to the user over time to improve the experience.

- **Examples:**
  - **Pantry Chef:** The app remembers what ingredients the user commonly has.
  - **Expense Tracker:** The app learns that a receipt from "Starbucks" always belongs to the "Coffee" category.
- **Implementation:** Using a local database (e.g., SQLite) to store user preferences or history.

FR5: User Feedback Loop

Users must be able to rate the accuracy of the AI output.

- **Implementation:** A "Thumbs Up/Thumbs Down" button or a star rating system for every AI prediction.
- **Purpose:** This data can be used for future retraining of the model (reinforcement learning).

5. Non-Functional Requirements

- **Usability:** The interface must be intuitive for a non-technical user.
- **Reliability:** The system should handle erroneous inputs without crashing.
- **Ethical Considerations:** Data privacy must be respected. The application must not promote bias or harmful stereotypes.

6. Deliverables

1. **Project Proposal (Due: February 20, 2026):** Outline of the problem, proposed solution, and dataset.
2. **Project Update (April 29, 2026):** In-class presentation discussing the functional backend model and basic UI.
3. **Final Report & Documentation (Due: May 22, 2026):**
   - System architecture diagram.
   - Model design and training justification.
   - Performance evaluation results.
4. **Final Presentation & Demo (May 25, 2026):** 10-minute live demonstration.

7. Evaluation Criteria

1. Problem Formulation & Scope (10 Points)

- **[5 pts] Problem Definition:** Clear, concise definition of the daily problem. It is practical, and the target user is well-defined.
- **[5 pts] AI Justification:** compelling argument why AI is required over a simple script (e.g., handles unstructured data, makes predictions, adapts to new data).

2. Data Engineering & Preprocessing (15 Points)

- **[5 pts] Data Quality & Quantity:** Dataset is appropriate for the complexity of the problem. Data is sourced legally/ethically.
- **[5 pts] Data Preprocessing:** Clear documentation of cleaning, normalization, handling missing values, or vectorization.
- **[5 pts] Data Augmentation/Splitting:** Proper use of augmentation to handle class imbalance and correct partitioning into Training/Validation/Testing sets to avoid data leakage.

3. Model Architecture & Implementation (20 Points)

- **[8 pts] Model Selection:** Justification for the chosen algorithm (e.g., CNN, RNN, LLM API). Does the model complexity match the problem?
- **[7 pts] Implementation:** Code is clean, well-commented, and technically sound. Model loads and predicts correctly.
- **[5 pts] Hyperparameter Tuning:** Evidence of attempts to optimize the model (learning rate, epochs, regularization).

4. Technical Evaluation (20 Points)

- **[10 pts] Performance Metrics:** Use of appropriate metrics (F1-Score, Accuracy, Precision, Recall, MSE) tailored to the project goal.
- **[5 pts] Visualization:** Inclusion of Confusion Matrices, ROC curves, or Loss/Accuracy graphs over epochs.
- **[5 pts] Latency Assessment:** Testing and reporting of inference time to ensure it meets the <2 second constraint.

5. Application Functionality & UX (20 Points)

- **[8 pts] Backend Integration:** Seamless connection between the UI and the AI model (e.g., API is functional, stable).
- **[7 pts] User Experience (UX):** The app is intuitive for non-technical users. It provides clear input and actionable output.
- **[5 pts] Functionality (FR1-FR5):** App implements input, immediate processing, feedback loops, and personalization features.

6. Documentation & Report (10 Points)

- **[5 pts] Structure & Detail:** Report follows the provided structure, is well-organized, and technically thorough.
- **[5 pts] Analysis & Future Work:** Honest reflection on model limitations, bias, privacy considerations, and concrete steps for improvement.

7. Demo & Presentation (5 Points)

- **[3 pts] Live Demonstration:** The demo runs successfully, showing a clear, high-value use case.
- **[2 pts] Technical Communication:** Ability to explain the technical approach and answer questions clearly.

Total Score: _____ / 100

AI Application Ideas

1. Smart Waste Sorter (Computer Vision)

- **Concept:** A mobile app that uses the camera to classify waste items as **recyclable**, **compostable**, or **landfill** and tells the user which bin to use.
- **Technical Focus:** Convolutional Neural Networks (CNN) for image classification.

2. Pantry Chef (NLP + Vision)

- **Concept:** The user takes a picture of the inside of their fridge/pantry. The app identifies the ingredients, suggests recipes using *only* those items, and generates a shopping list for missing ingredients.
- **Technical Focus:** Object detection (Vision), text generation via LLM API (NLP).

3. Smart Posture Coach (Computer Vision)

- **Concept:** A desktop app that uses the webcam to monitor a user's posture while working. It alerts the user when they are slouching and suggests stretching exercises.
- **Technical Focus:** Pose estimation using libraries like MediaPipe or OpenPose.

4. Intelligent Mail Triage (NLP)

- **Concept:** A tool that analyzes incoming emails and automatically drafts replies, summarizes long threads, or flags urgent messages based on the user's past behavior.
- **Technical Focus:** Sentiment analysis, text summarization, classification.

5. AI Expense Tracker (NLP + OCR)

- **Concept:** User takes a picture of a paper receipt. The app extracts the store name, date, total amount, and categorizes the expense automatically.

- **Technical Focus:** Optical Character Recognition (OCR), Named Entity Recognition (NER).

6. Voice-Powered Home Inventory (NLP)

- **Concept:** Users verbally list items they are putting into storage ("Putting jacket in box 4 in the attic"). Later, they can ask, "Where is my winter coat?"
- **Technical Focus:** Speech-to-Text (STT), conversational agent.

7. Personalized News Summarizer (NLP)

- **Concept:** The app tracks topics of interest to the user, scrapes news articles, and generates a custom, concise daily briefing podcast or text summary.
- **Technical Focus:** Web scraping, LLM-based text summarization.

8. Accessible Scene Descriptor (CV + TTS)

- **Concept:** An app for visually impaired users that captures a scene via the phone camera and describes it aloud (e.g., "A red car is approaching from the left").
- **Technical Focus:** Image captioning, Text-to-Speech (TTS).

9. Smart Study Planner (Reinforcement Learning/ML)

- **Concept:** Users input their exams and subjects. The app builds a study schedule. As the user completes tasks, the AI adjusts the schedule based on what subjects the user finds hardest.
- **Technical Focus:** Recommendation engine, adaptive learning algorithms.

10. AI Stylist (Computer Vision)

- **Concept:** User uploads photos of their clothes. The app categorizes them and suggests outfits based on the local weather forecast.
- **Technical Focus:** Image classification, data mapping.

AI Project Proposal: [Project Name]

Team Members: [Names]

Course: [Course Name]

Date: [Submission Date]

---

1. Project Overview

- **Application Name:**
- **Theme/Category:** (e.g., Health, Productivity, Sustainability)
- **Core AI Technology:** (e.g., Computer Vision, NLP, Machine Learning)

2. Problem Statement

- **What is the specific, everyday problem you are trying to solve?**
- **Who is the target user?**
- **Why is AI the right tool to solve this problem?** (i.e., Why can't a simple procedural script solve this?)

3. Proposed Solution

- **Describe the proposed application in detail.**
- **What is the expected user workflow?** (e.g., The user takes a picture → AI processes it → User receives actionable advice).

4. Technical Approach

4.1. Data Sourcing

- **What dataset will you use?** (Provide a link if using a public dataset like Kaggle or UCI).
- **If collecting your own data, how will you do it?**
- **What is the size of the dataset?**

4.2. Model Architecture

- **What model(s) do you plan to use?** (e.g., Pre-trained CNN like ResNet, Transformer model for text, Scikit-learn Random Forest).
- **Why did you choose this model?**

4.3. Evaluation Metrics

- **How will you measure success?** (e.g., Accuracy, Precision, Recall, MSE).

5. Potential Challenges

- **What are the biggest technical risks?** (e.g., Limited data, high inference latency, difficulty in preprocessing).

6. Project Timeline (Milestones)

- **Data Collection & Cleaning (Due Date):**
- **Model Training & Evaluation (Due Date):**
- **UI Development & Integration (Due Date):**
- **Final Report & Demo Preparation (Due Date):**

Final Project Report: [Project Name]

Team Members: [Names]

Course: [Course Name]

Date: [Submission Date]

---

1. Introduction

- **Problem Definition:** Reiterate the everyday problem and its significance.
- **Objective:** Clearly state what the application achieves.
- **Project Scope:** Briefly outline the features included in the final prototype.

2. Related Work

- Briefly discuss any existing solutions to this problem and how your AI-based approach differs or improves upon them.

3. Data Acquisition and Preprocessing

- **Dataset Description:** Source, size, features, and target labels.
- **Data Preprocessing Pipeline:**
    - Cleaning (handling missing values, outliers).
    - Augmentation techniques (if applicable).
    - Normalization/Vectorization methods.
- **Data Split:** Details on training, validation, and testing sets.

4. Methodology & Technical Approach

- **System Architecture:** A high-level diagram showing how the frontend, backend, and AI model interact.
- **Model Selection:** Justification for the chosen algorithm(s) (e.g., CNN, RNN, Random Forest).
- **Training Process:**
    - Hyperparameters used (learning rate, epochs, batch size, etc.).
    - Training environment (GPU/CPU used).
    - Techniques to avoid overfitting (dropout, regularization, early stopping).

5. Evaluation and Results

- **Metrics:** Definition of metrics used (Accuracy, Precision, Recall, F1, MSE, etc.).

- **Results:** Present findings using tables and graphs (e.g., Confusion Matrix, ROC curve, Loss vs. Epoch plot).
- **Discussion:** Analyze the performance. Did the model meet the latency requirements (< 2s)?

6. User Interface and Integration

- **UI Design:** Screenshots of the application and description of user flow.
- **Integration:** How the model was deployed within the app (e.g., Flask API, TensorFlow.js, etc.).

7. Ethical Considerations and Limitations

- **Limitations:** What can the model *not* do? What are its failure points?
- **Ethical Considerations:** Discussion on data privacy, potential biases in the training data, and environmental impact of training.

8. Conclusion and Future Work

- Summary of achievements.
- Potential improvements for future iterations (e.g., deploying to cloud, increasing dataset size, improving UI).

9. References

- Cite all datasets, libraries, pre-trained models, and academic papers used.

Final Presentation & Demo Script Template

Time Limit: 10 Minutes

---

1. Introduction (1 Minute)

- **Hook:** Briefly state the everyday problem (e.g., "Have you ever thrown away food because you forgot it was in the back of the fridge?")
- **The Solution:** Introduce your app name and its core purpose.
- **Value Proposition:** What is the primary benefit to the user?

2. The Problem & User Need (1 Minute)

- Describe the target user persona.
- Explain why existing solutions (non-AI) are inadequate.

3. Technical Approach: AI Model (3 Minutes)

- **Data:** Briefly describe the dataset used to train the model.
- **Architecture:** Explain *why* you chose this specific AI approach (CV, NLP, ML) and model architecture.
- **Training:** Mention any interesting challenges faced during training (e.g., overfitting, data imbalance) and how you solved them.

4. Live Demonstration (3 Minutes)

- **Scenario:** Walk through a realistic user scenario.
- **Action:** Show the user inputting data (uploading photo, speaking, typing).
- **Reaction:** Show the AI processing the input and displaying the result.
- **Speed:** Highlight that the inference time meets requirements (< 2s).

5. Evaluation & Results (1 Minute)

- **Metrics:** State the final accuracy/performance metrics (F1-score, MSE, etc.).
- **Validation:** Briefly show a confusion matrix or key graph to prove model reliability.

6. Conclusion & Future Work (1 Minute)

- **Summary:** Reiterate how the app solves the everyday problem.
- **Limitations:** Honestly mention what the model cannot do well.
- **Future Scope:** What is one feature you would add next?
- **Q&A:** Open floor for questions.